THE EBOOK · 12 CHAPTERS · ABOUT 25 MINUTES

Rehearsing a lakehouse before the real data arrives

A measured rehearsal of a complete lakehouse: what the team built, how it prevented bad data reaching reports, what it cost, and what must change before production.

> **Reading note.** This is a fictional operator and synthetic data. Numbers marked as measured came from the rehearsal; costs are labelled either Azure charges or Databricks list-price usage. Open controls are stated rather than hidden.

1. The brief and the budget
2. Synthetic data that misbehaves
3. The architecture
4. Landing without duplicates
5. Silver rules as data
6. Gold and the 53-second summaries
7. Reconciliation first
8. What it cost
9. Dev and prod
10. Catalogue and reports
11. Lessons for the real project
12. Epilogue: switching it off

Chapter 1

The brief and the budget

Lumen Ridge runs an online casino and sportsbook, a wallet and payments stack, and a few hundred retail betting shops with cashiers, terminals and shifts. The real platform will pull from seven sources: five APIs, one PostgreSQL database and one file feed. Finance needs internal figures to agree with gateways and banks to within 0.1 percent, with every difference explained.

The constraint that shaped everything was money: a USD 200 monthly budget, email alerts at USD 100, 150, 176, 190 and 200, and an automated kill switch that stops all compute at USD 190. Every design choice in this book starts from that number.

> Rule zero: default to the cheapest safe option, and state the cost of anything new before building it.

Chapter 2

Synthetic data that misbehaves

Clean demo data proves nothing. Two generators produced a "dirty" month instead: late-arriving rows, exact re-sends of whole files, two different versions of a bet with the same timestamp, settlements that never arrive, and reference snapshots that change under the facts. The online generator is fingerprinted, so data from different code versions can never be mixed in one load.

DATASET	DAYS	ROWS LANDED	BUILT WITH
Online casino, sportsbook, wallet, payments	30	106.7M	Notebooks (Bronze, Silver, Gold)
Retail slips, bets, cash, floats, shifts	about 30	114.9M	COPY INTO + dbt
Gold daily summaries	all	about 163k	dbt tables

Chapter 3

The architecture

The platform is a medallion lakehouse in Azure Databricks with Unity Catalog. Bronze keeps every delivered version, never updated. Silver keeps the latest valid version per key and sends everything else to quarantine with a reason. Gold is a star schema with declared grains, and money lives on exactly one fact per process so joins cannot double count. All compute is serverless and stops when idle.

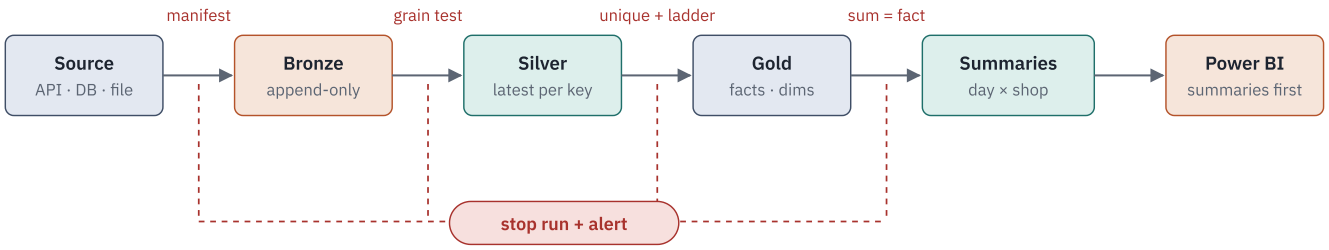


Figure 1. One load, layer by layer. Each gate fails closed: a failed check stops the run before Gold or any report changes.

Source: rehearsal control design. This is the target operating model; remaining production gates are listed in the evidence note.

The tools, and the job each one does

Consistency comes from the control rules and their evidence, not from forcing every workload through one product. This is the toolchain used in the rehearsal and the intended responsibility of each part.

TOOL	ROLE IN THE REHEARSAL	BOUNDARY
Azure Databricks + Delta Lake	Serverless compute and durable Bronze, Silver and Gold tables.	Stops when idle; cluster use needs a measured exception.
Unity Catalog	Catalogs, access rules, lineage, audit metadata, tags and personal-data controls.	One catalog and storage boundary per environment.
Lakeflow Jobs	Runs the retail landing step before its transformation task, and schedules controlled workloads.	A failed upstream task prevents the next one from starting.
dbt	Retail Silver and Gold transformations, tests and daily summary models.	Only approved models become physical tables; single-use intermediates stay ephemeral or views.
Python and SQL notebooks	Online ingestion, Silver, Gold and reconciliation.	dbt work for online remains a comparison experiment, not a production dependency.
GitHub CI/CD + Terraform	Reviews code, contracts, tests, job configuration and infrastructure plans.	Promotes code and configuration only; never raw development data.
Power BI	Consumes Gold aggregates first; detail facts support bounded drill-through.	Reconciles report totals to Gold and releases its warehouse after validation.

Chapter 4

Landing without duplicates

Every delivery arrives with a manifest: file checksum, size and row count. Nothing lands until all three match. Landing uses `COPY INTO`, which skips files it has already loaded, so a re-sent delivery cannot land twice. Every Bronze row carries its run id, source file, schema version and ingest time.

The rule that overrides everything else: Bronze may keep re-sent rows as audit history, but Silver and Gold hold exactly one row per declared grain. Each trusted Silver model declares its business key, compound keys included, and a `unique_grain` test fails the build before Gold is touched. A local run of 25 grain checks across Silver and Gold found zero duplicate keys, on data that deliberately contained re-sends.

Chapter 5

Silver rules as data

Retail Silver is generated from a rules file rather than hand-written: types, allowed values, parent checks and version rules live as data, and a script writes 58 dbt models from them. The latest version per key wins by change time, then ingest time, then payload hash. If the same key arrives twice with the same change time and different content, both versions go to quarantine as `CONFLICTING_DUPLICATE`: a silent pick would hide a source problem.

Only tables that earn their storage are tables. A small check fails the build if a dbt model becomes a table without a written reason, which turned eleven unnecessary intermediate tables into views.

Chapter 6

Gold and the 53-second summaries

The first Power BI refresh against the row-level facts ran for more than twenty minutes and kept the warehouse awake the whole time. The fix was upstream: seven daily summary tables in Gold (casino, sportsbook, wallet and payments by day and operator; retail bets, cash and shifts by day and shop). They built in 53 seconds for about four US cents, and tests prove each one equals its fact exactly in rows and money.

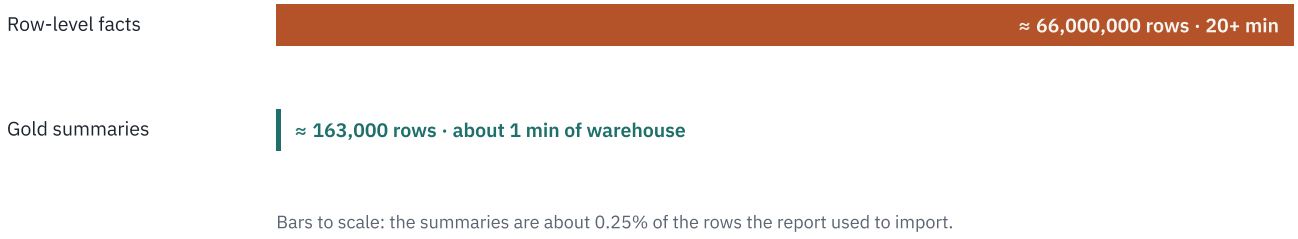


Figure 2. The biggest cost win of the rehearsal came from moving aggregation upstream into Gold.

Source: seven measured summary tables. Row counts are approximate and represent the rehearsal date range.

Reconciliation first

Reconciliation is not a report at the end; it is a ladder of checks at every hop, each writing to an append-only results table with a cause code. When figures stop agreeing, the first failing rung tells you where to look.

RUNG	COMPARES	TYPICAL CAUSE CODES
Source → file	manifest rows, money, checksum	SHORT_READ, HASH_MISMATCH
File → Bronze	rows and money landed	NOT_LANDED, DUPLICATE_LOAD
Bronze → Silver	accepted + quarantined + duplicates = Bronze keys	QUARANTINED, NOT_LOADED_TO_SILVER
Silver → Gold	key by key, money exact	MISSING_IN_GOLD, VALUE_MISMATCH
Gold → report	summary totals vs facts for the same window	REPORT_MISMATCH
Business rules	casino vs wallet, payment vs settlement, cash vs shift	MISSING_IN_WALLET, SETTLEMENT_NOT_DUE

For the online month, Silver against Gold matched on all four checks. The cross-system breaks that remained were the ones the generator planted, each explained by a cause code.

Chapter 8

What it cost, and why the bill disagreed

Databricks reported USD 45.87 at list price for the first five days. Azure billed USD 5.89. The workspace was on a 14-day trial, so compute was free on the invoice, and the real charges were the network gateway (USD 4.74), a public IP and storage. The lesson: list-price usage tells you what drove cost; the cloud bill tells you what you pay. Track both, and decide what happens when the trial ends.

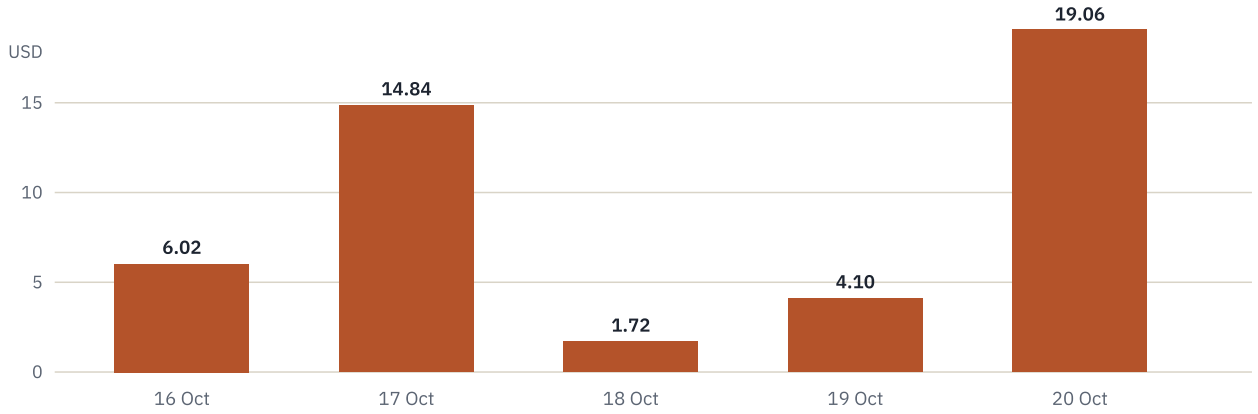


Figure 3. Databricks spend at list price per day. 20 October was the 78-million-row online load in four batches, with reconciliation after each and full Power BI refreshes.

Source: Databricks usage data, 16–20 October 2026. Azure charges shown separately were USD 5.89 during the first week; the two values are not directly comparable.

Where the money went on the expensive day

Four separate batches each paid their own start-up, reconciliation ran after every batch instead of once, and Power BI imported the full row-level facts. The rules since then: larger batches, light checks per batch with the full ladder once at the end, summaries first in Power BI, and the warehouse stopped straight after validation.

Chapter 9

Dev and prod

The rehearsal ran in one workspace. The real project will use two, `dev` and `prod`, each with its own catalog bound only to its own workspace, its own storage, Key Vault and service principal. Only code moves between them, through a reviewed pull request and an approval gate. Data never moves from dev to prod; prod rebuilds from its own Bronze. Dev works on synthetic or masked samples only.

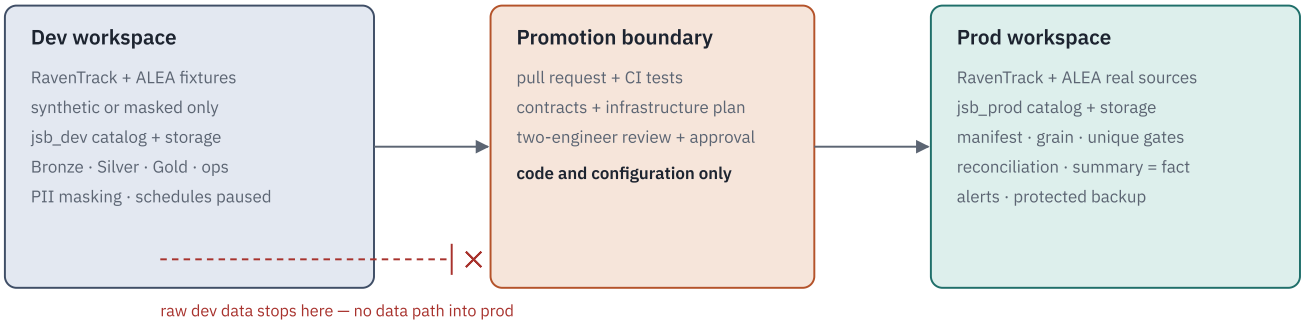


Figure 4. Dev and prod have separate workspaces, catalogs and storage. Production reads its own RavenTrack and ALEA sources; only reviewed code and configuration cross the promotion boundary.

Target design for the real project. Private networking and source connectivity remain security-design decisions.

Cost note: a second workspace with private networking adds a fixed network gateway charge of about USD 35 a month even when idle, so the decision belongs at project start, as a line item.

Chapter 10

Catalogue and reports

A single YAML file describes every table and column once: owner, domain, grain, source, refresh and personal-data class. A script turns it into Unity Catalog comments and tags and a live catalogue view that also shows each table's latest quality failure rate, reconciliation status and freshness. The first run described 171 objects and 2,120 columns and tagged 17 objects as holding personal data. A business glossary links each term, such as GGR, to the columns that implement it.

Both Power BI reports are generated as text projects, statically validated before anyone opens them, and read the Gold summaries by default. Detail tables load only a bounded date window, and a Reconciliation page shows the summaries matching the detail for the selected days.

Chapter 11

Lessons for the real project

1. **Aggregate upstream.** Summaries in Gold were the single largest cost and speed win.
2. **Integrity is a gate, not a report.** Declare the grain of every Silver model and fail the build on a duplicate key.
3. **Reconcile every hop** and record the first failing layer, with cause codes and history.
4. **Serverless by default.** A classic cluster only on a measured need, isolated to one task.
5. **List price is not the bill.** Use the cloud bill for totals and usage tables for attribution; watch fixed network costs.
6. **API sources cost waiting time, not rows.** Measure throughput with a capped probe before any backfill.
7. **Small tooling traps are real:** shells that rewrite paths, quotes stripped from arguments, partition columns missing from Parquet files, hashes that turn NULL, metadata changes that conflict in parallel.
8. **Two reviewers beat one.** Every change was argued in a pull request until both wrote AGREE; disagreements twice on one point went to a human.

Evidence and limits

Measured in the rehearsal: 221.6M synthetic rows landed; zero duplicate keys in 25 tested retail Silver and Gold grains; seven Gold summaries built in 53 seconds; their row and money totals matched the related facts. The online Silver-to-Gold checks also matched on the tested controls.

Not yet production evidence: real RavenTrack and ALEA source contracts, Bronze raw-payload/hash metadata, tombstone and replay tests, translation drift tests, a shared API runner and operations schema, retail masking, and the complete first-failing-layer reconciliation ladder. These remain release gates before real data is trusted.

Cost limit: Azure charges and Databricks list-price usage are separate measures. Trial credits can hide compute on an invoice; network and storage charges remain visible in Azure Cost Management.

Chapter 12

Epilogue: switching it off

A rehearsal should end on purpose. A scheduled runbook deletes the Databricks workspace one day before the trial ends, or immediately if spend ever reaches the full budget, whichever comes first. The network gateway goes with it. The data stays in storage, the code stays in git, and the whole platform can be rebuilt from Terraform in about an hour when the real project begins.

Figures

- Figure 1. One load with fail-closed gates
- Figure 2. Rows imported before and after summaries
- Figure 3. Spend per day at list price
- Figure 4. Dev to prod promotion barrier

Index and glossary

Bronze. Immutable delivered data and run metadata. Chapter 4

Contract. The agreed source, schema, key, freshness and deletion rules. Chapter 3

Data quality gate. A test that stops a failing run before it can update the next layer. Figure 1

GGR. Gross gaming revenue, linked in the catalogue to its implementing columns. Chapter 10

Gold. Business-ready facts, dimensions and approved aggregates. Chapter 6

Grain. The one row that a table is allowed to represent; its declared key prevents double counting. Chapter 4

Manifest. Delivery identity: file checksum, size and row count. Chapter 4

Quarantine. Invalid or conflicting records held with a reason instead of silently chosen or discarded. Chapter 5

Reconciliation ladder. Source-to-report checks that expose the first failing layer. Chapter 7

Silver. Typed, validated, deduplicated data with personal-data controls. Chapter 5

Summary table. A Gold aggregate used first by Power BI to cut scan time and cost. Chapter 6

Watermark. A saved source cursor, read with an overlap window to catch late changes. Chapter 4

Lumen Ridge is fictional; all data is synthetic. Written by Claude, reviewed by Codex. Suggestions welcome as GitHub issues.